NetEdit: An Orchestration Platform for eBPF Network Functions at Scale

Theophilus A. Benson‡ Prashanth Kannan⋆ Prankur Gupta⋆
Balasubramanian Madhavan⋆ Kumar Saurabh Arora⋆ Jie Meng⋆
Martin Lau⋆ Abhishek Dhamija⋆ Rajiv Krishnamurthy⋆
Srikanth Sundaresan⋆ Neil Spring⋆ Ying Zhang⋆
‡Carnegie Mellon University ⋆Meta

ABSTRACT

Managing the performance of thousands of services across millions of servers demands a networking stack that can dynamically adjust protocol settings to match diverse priorities and network characteristics. Moreover, given the constantly evolving nature of services and their requirements, the set of configurable protocols must remain adaptable. However, current host networking stacks lack the necessary flexibility and adaptability. Although eBPF shows promise in this regard, it lacks essential primitives for efficient development and safe deployment of multiple co-existing services.

This paper presents our experience developing NetEdit, a system that orchestrates the composition, deployment, and life-cycle management of eBPF programs across a large fleet of servers at Meta. Our design offers a unified abstraction for various kernel hook-points, decouples policies from programs using a rich configuration language, supports explicit object management for reliable deployment and provides extensive testing methods. NetEdit has been deployed in our production network for five years and now supports thirteen Network Function applications. We have observed that NetEdit-enabled functions improve average service performance by 3X and network performance by 4.6X, showcasing its significant real-world impact.

CCS CONCEPTS

• **Networks → Network management**; **Programmable networks**; **Network design principles**.

KEYWORDS

eBPF; Network Management; Orchestration

ACM Reference Format:
Theophilus A. Benson‡ Prashanth Kannan⋆ Prankur Gupta⋆, Balasubramanian Madhavan⋆ Kumar Saurabh Arora⋆ Jie Meng⋆, Martin Lau⋆ Abhishek Dhamija⋆ Rajiv Krishnamurthy⋆, Srikanth Sundaresan⋆ Neil Spring⋆ Ying Zhang⋆, ‡Carnegie Mellon University ⋆Meta. 2024. NetEdit: An Orchestration Platform for eBPF Network Functions at Scale. In *ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24), August 4–8, 2024, Sydney, NSW, Australia.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3651890.3672227

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0614-1/24/08
https://doi.org/10.1145/3651890.3672227

1 INTRODUCTION

Optimizing network performance on a massive scale, encompassing tens of billions of requests across tens of thousands of services distributed among millions of containers, poses an exceptionally complex challenge. At Meta, we tackle this challenge by fine-tuning the host networking stack and network protocols based on the distinct characteristics of our network, hardware, and services.

Although various options exist for tuning host networking stacks, eBPF [23, 37] has emerged as a widely adopted approach [7, 8, 14, 17, 21, 24, 51]. Its popularity stems from its ability to dynamically introduce functionality into the kernel, providing unparalleled flexibility and fine-grained control without sacrificing infrastructure velocity or the ability to upgrade our kernels. Despite widespread adoption in industry, eBPF has primarily been used for isolated, on-demand tasks such as enabling observability [22, 50, 57**?**] for diagnosis and security or crafting tightly integrated applications like Isovalent's Cilium[8] and Meta's Katran[17]. These deployments typically have low code velocity due to their specific, often single-purpose nature. However, developing, deploying, and configuring arbitrary eBPF programs at high velocity to tailor host network stacks to services and hardware is challenging. The current eBPF ecosystem is ill-equipped for highly dynamic settings and lacks the high availability guarantees and rigorous testing capabilities that we require.

What we envisioned—and implemented—was NetEdit, an ecosystem of abstractions, interfaces, and libraries that enables continuous development, release, and management of a large set of production-grade eBPF applications. We define a set of eBPF programs that collectively achieve a specific high-level network function as a *tuningFeature* – in NetEdit these eBPF programs collectively tune or tailor aspects of the network stack to a service. Examples include new congestion control protocols [16] or fine-tuning window sizes for TCP connections, more are shown in Table 3.

NetEdit assists the development and orchestration of tuningFeatures across heterogeneous infrastructure in a highly available, persistent, and safe manner. To achieve this, we overcame the following five challenges, resulting in what we believe to be the first production-grade eBPF orchestration platform for networking use cases at scale.

- C1 (Diverse and changing kernel hook-points) The various eBPF interfaces and capabilities exposed by the kernel across different hook-points evolve rapidly with new releases[6, 40, 49]. The complexity of managing existing applications and deploying new ones across multiple kernels—a reality in

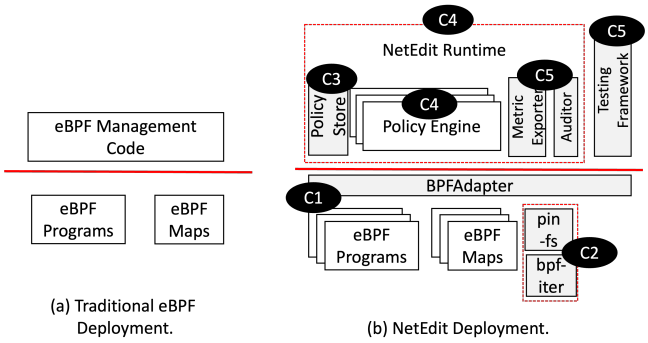

Figure 1: eBPF Deployment: (a) Traditional, (b) NetEdit (The black circles map NetEdit's challenges to its components).

our fleet—led us to *decouple applications from kernel hook-points*. We provide a *uniform interface across eBPF program and kernel versions*, simplifying the management of arbitrary applications (Section 5.1).

- C2 (Disruption during upgrades): Production environments are volatile, even for eBPF infrastructures. During upgrades, we expect our eBPF programs to continue functioning even in the face of failures. However, user space restarts during upgrades can introduce undesirable intermediate states, such as the kernel incorrectly removing eBPF programs or objects. Our approach is *to provide explicit control over the creation and deletion of eBPF objects* (Section 5.2).
- C3 (Complex and adaptive configurations): An eBPF program can be applied to different network traffic depending on service requirements and network conditions, both of which can change rapidly. Static, hard-coded policies may quickly become stale as the network environment changes, resulting in suboptimal or even incorrect behavior. Moreover, policies can be complex combinations of diverse conditions such as flow statistics, server, and application. Our approach is a flexible policy configuration language capable of expressing specific requirements regarding locality and granularity, *efficiently translating policies into the rigid constructs supported by eBPF programs* (Section 6.1).
- C4 (Dynamic service placement): eBPF programs may be tailored to specific infrastructure, which couples eBPF functionality with infrastructure characteristics (e.g., hardware). However, the placement of services is dynamic and agnostic to eBPF program compatibility. Our approach is to detect the current state of our infrastructure, in a distributed manner, to determine the appropriate placement of eBPF programs and their configurations (Section 6.3).
- C5 (Correlated kernel bugs): While eBPF programs alone can be verified, we have encountered several unknown bugs under certain kernel-eBPF interactions. To improve safety, we establish a testing framework to rigorously validate the eBPF-kernel interfaces by *isolating eBPF programs* (Section 6.4 and Section 9.1).

To summarize, NetEdit addresses the aforementioned issues through *separation of responsibilities*; we highlight its key innovations in Fig 1. Analogous to SDN, NetEdit introduces a data plane: a uniform abstraction that hides kernel-specific eBPF implementation and exposes simple eBPF object life cycle management actions (C1-Fig 1), overriding the kernel's default eBPF management behavior and providing availability and correctness guarantees (C2-Fig 1). The data plane enables a highly decentralized control plane, an offline, expressive, highly flexible domain-specific policy configuration language, which maps high level business intent into match + action key-value pairs that can be enforced within eBPF language constraints (C3-Fig 1). Our domain-specific configuration language allows defining policies at various granularity, locality, and scope, thus enabling a broad range of operational tasks ranging from maintenance, debugging, and staged-code rollouts as well as deployment decisions about when to enable or disable eBPF programs (C4-Fig 1). Additionally, NetEdit includes designs that to enable high availability observability and to seamlessly integrate eBPF into our existing testing framework (C5-Fig 1).

NetEdit has been deployed in our large production network for over 5 years and it has grown to support 13 diverse tuningFeatures, ranging from novel congestion control protocols to address translation proxies. We have found that NetEdit tuningFeatures have improved average service performance (3X for a storage service and 2.5X for an MLInference service) and improved network performance (reducing link loss by 4.6X and transport retransmits by 4.5X) (Section 8.2). In addition, we share the operational experiences, including various bugs uncovered (e.g., [34]), which results in patches and lessons shared with the broader eBPF community (e.g., [30–33, 35]) (Section 9.3).

2 BACKGROUND AND MOTIVATION

Next, we provide an overview of eBPF (Section 2.1) and then discuss NetEdit's motivation (Section 2.2), and requirements (Section 2.3).

2.1 eBPF for Network Tuning

An eBPF program is a stateless code snippet guaranteed to be executed by the kernel when it reaches a specific "hook-point" or line in the kernel. eBPF has well-defined boundaries, such as a limited code size and complexity, enabling the kernel to validate and verify the code before loading. To maintain state between executions of the same program or share state between programs, an eBPF program must utilize external data structures called "eBPF maps", which are also accessible by userspace programs through system calls.

At Meta, we chose eBPF for networking tuning due to several advantages over traditional approaches such as SetSock, IPTables, and kernel modules. Firstly, eBPF offers transparency as it does not require changes to the application code (unlike SetSockOpt). Secondly, eBPF allows for better adaptation to kernel evolution, whereas kernel modules may need to be rewritten during certain kernel upgrades. Lastly, eBPF provides control over a wide range of networking functionality, e.g., matching based on criteria like port number, NIC model, etc, which is not supported by IPTables. In contrast, other hyperscalers like Google achieve flexible networking tuning by adopting userspace stacks [36]. However, this approach often requires complex optimizations in the kernels and even specialized network cards. We provide a detailed comparison of these approaches in Appendix A.

	ML_Service	Storage
avg	3	2
p95	13	5

Table 1: The number of active tuningFeatures for two services in our fleet: average and 95th percentile.

2.2 Motivating Example

We illustrate our challenges using a real-world common use case within our fleet. Service A requires two tuningFeatures: 1) a TCP window tuner that mitigates bursts by limiting receiver window sizes based on round-trip time (RTT) and traffic patterns, and 2) a specific congestion control algorithm, Cubic, applied only to cross-region flows with high RTT to ensure fairness and optimal performance. Both tuningFeatures involve flow classification based on RTT, but they require different configurations (window size versus congestion control algorithm). In this example, Service A is migrated from one physical server to another with a different NIC speed. In such cases, while the congestion control tuning may remain the same, the window settings need to be adjusted; the old window size on a faster NIC can result in lower bandwidth, while on a slower NIC, it can lead to uncontrolled bursts resulting in buffer buildup and contention in the ToR switch [19].

Concurrently, Service B replaces Service A on the old host, and requires a tuningFeature that sets a specific TCP initial window value to improve startup latency performance. Furthermore, over weeks or months, the traffic profiles of both Service A and B may change significantly, requiring updates to their window configurations. These dynamic changes highlight the need for flexible and adaptive window configurations to accommodate varying NIC speeds, evolving traffic patterns, and specific performance requirements.

We further quantify the dynamism of configuration changes by examining the number of active tuningFeatures for two services over a one-month period – a storage and an ML_Service. Table 1 illustrates a significant variation between the average and the p95 for each service. Additionally, we observe significant differences across these services. Given that each physical server hosts multiple such services, the actual number of tuningFeatures varies, and some hosts experience more frequent changes than others. There is no discernible pattern to these changes. Both examples highlight several key requirements for managing programs for network configuration at our scale.

2.3 Requirements

Next, we elaborate on NetEdit's requirements and tie them back to the aforementioned five challenges.

Development Transparency (C1): Each hook-point in the kernel exposes a distinct, heterogeneous set of primitives and interfaces for manipulating and managing programs (e.g., attaching, detaching, or loading programs). Consequently, developers must write significant management logic and "glue-code" to orchestrate functionality across different hook-points. To enhance development efficiency and maintainability, we *need a systematic approach to manage these diverse hook-points.*

Highly-Available Functionality (C2): Currently, the kernel automatically manages all eBPF objects (programs and maps), tracking references and automatically removing objects when they are no longer referenced. However, in certain scenarios, such as when the user space application fails or reboots, the kernel may remove objects, including those containing critical state. The behavior is undesirable as we require constant network tuning. Additionally, while the kernel triggers programs at hook-points, newly installed or modified programs need the ability to retroactively apply configurations without relying on specific events. For example, the initial congestion window or a specific congestion control algorithm settings are usually triggered early in the connection set up, but we need the ability to set these for long-lasting connections that span multiple eBPF program restarts [1]. These issues highlight *the need for explicit control over the object life cycle (deletion, triggering) based on external contextual knowledge about the current operating conditions.*

Safe and Correct Deployments (C5): While the verifier ensures the safety of individual programs, there is a lack of comprehensive tools for understanding, testing, and debugging the eBPF-kernel interface and ensuring the correctness of programs in large-scale production settings, especially when multiple programs operate on the same connection or packet state. This challenge is amplified by our diverse use cases and the need for explicit control, which often involves exploring uncommon interfaces and control paths. For instance, consider the scenario where multiple programs operate on the same connection or packet state, requiring careful coordination and synchronization. Consequently, *there is a pressing need for extensive and robust frameworks to efficiently test eBPF-kernel interfaces, uncover potential bugs, and prevent outages.*

Context Specific Orchestration (C3/C4): Currently, the eBPF ecosystem lacks mechanisms that enable dynamic determination of which programs to deploy or configure based on the services running on a server. Deployment is either manual (e.g., for on-demand debugging) or identical across hosts. We need a mechanism to hide infrastructure, service elasticity, and migration details by dynamically adjusting the deployment and configuration of various programs to match infrastructure changes. Moreover, while tuning configurations are highly specialized and tailored to address underlying hardware, service, and traffic heterogeneity, the policy language should be sufficiently general to minimize the number of policies required and the burden of configuring a large production environment. To ensure these programs support service (e.g., scaling/migration) and infrastructure (e.g., server failure) dynamicity, *we require highly dynamic, quick, and scalable orchestration of program life cycles.*

3 COMPARISON OF OPENSOURCE EBPF ORCHESTRATORS

We acknowledge that NetEdit's scale of managing a diverse features across a fleet of million of servers is potentially not unique, hyperscalers should face similar challenges, however, NetEdit is unique in its early adoption of eBPF and goal to provide tuningFeatures tailored to our unique services and infrastructure. Moreover

[1]Recall, eBPF programs are stateless and reactive and will not retroactively enforce tuning policies

NetEdit has been in production for the last six years, while recently opensourced eBPF managers and orchestrators, within the last 2-3 years, build on the kernel changes that we have up-streamed and potentially benefit from the documentation that we have published.

In Table 2, we briefly summarize their characteristics along our design goals. We observe that decoupling BPF-kernel is a fundamental issue any platform needs to overcome to practically use eBPF at scale. However the support and richness of the eBPF-kernel decoupling varies highly. While all support interface decouple, none support BPF-to-BPF decoupling. Important to note that while these efforts aim to support BPF-2-BPF interactions they rely on the kernel default primitives which provides inconsistent guarantees across hookpoints and versions. More generally, while there is some preliminary efforts to decouple resource management heuristics, they either use primitives we have previously open-sourced (i.e., BPF-iter), do not support some heuristics (e.g., decoupling observability) or do not implement in a method that aligns with our infrastructure (i.e., Warm-Pinning)

A key aspect of NetEdit is to enable global coordination of our tuningFeatures with the life-cycle of the applications that they augment. For example, turn-on or turn-off tuningFeatures on a host based on the lifecycle of services and even traffic on that host. This aspect is missing from all orchestrator (except Cilium) which is naturally given their emphasis on single node management. However, Cilium does not provide the lazy loading or scoping abstractions required for CPU efficiency and correct policy enforcement.

Finally, we introduce several Meta-specific capabilities to simplify and ease development of tuningFeatures which are missing from the frameworks in Table 2. Moreover, orthogonal efforts to simplify eBPF development are restricted in attach-points [5] or lack our operational context.

Open Source Systems	Decoupling BPF-kernel (Section 5)			Dynamic Integrated Lifecycle (Section 6)
	Interface (Section 5.1)	Heuristics (Section 5.2)	BPF-to-BPF (Section 5.3)	
DataDog [15]	Y	Y*	N	N
bpfd [11]	Y	Y*	N	N
l3af [54]	Y	Y*	N	N
Cilium [8]	Y	Y*	N	N
PolyCube [39]	Y	Y*	N	N
NetEdit	Y	Y	Y	Y

Table 2: Comparison of Opensource eBPF Cluster Managers. Existing frameworks do not support decoupling of eBPF-to-eBPF and do not provide dynamic integration eBPF with the application life-cycle. Cilium provides a static integration because it aims to be an always present sidecar. They only support a fraction of the heuristic decoupling that NetEdit requires.

4 NETEDIT DESIGN

We share the design principles that allow us to overcome the inherent limitations of today's eBPF ecosystem.

4.1 Design Principles

- **Decouple eBPF from the Kernel Layer:** eBPF interacts directly with kernel primitives and is governed by the kernel's resource management heuristics. However, these change frequently, provide inconsistent behavior, and vary widely across versions. To address these challenges, we decoupled eBPF from the kernel primitives, interfaces, and heuristics to enable uniform and predictable management of eBPF programs. The decoupling and the direct control allow us to support future use cases with minimal concern about the default kernel behavior.
- **Integrated Lifecycle Management:** eBPF programs transparently augment microservices; they generally lack visibility into cluster management events that impact service availability, such as migration, scaling, and reboots. To address this challenge, we have developed an API that integrates application lifecycle events into eBPF program management. This API includes a set of primitives that enable effective coordination between the service and the tuningFeatures lifecycles.
- **Fail-open:** Like any hyperscaler, we regularly update and redeploy our services, including NetEdit. However, NetEdit and its tuningFeatures must maintain the illusion of constant availability during updates and failures. This means that tuningFeatures continue tuning traffic and developers be able to introspect and debug an eBPF program even when NetEdit is unavailable, i.e., due to reboots or failures.
- **Minimal overheads:** To support latency-sensitive services, such as stream processing, ranking, recommendation systems, ML inference, and frontend services, we need to design NetEdit and its tuningFeatures with efficient resource usage in mind.

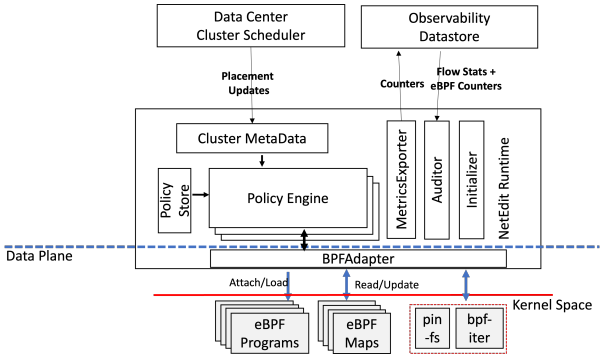

Figure 2: NetEdit Ecosystem.

4.2 Overview

NetEdit serves as: a) a cluster management system for an ecosystem of eBPF programs across a fleet of large-scale shared hardware, b) a development platform, providing reusable building blocks and testing capabilities for these eBPF programs, and c) a policy configuration engine, translating service requirements under dynamic conditions into correct deployments. At a high-level, its design consists of a data-plane that decouples the kernel from eBPF management, and a control-plane that manages and dynamically translates policy into primitives understood by eBPF.

```
class BpfAdapter {
  //BPF program is stored in binary as text
  BpfAdapter(adapterName, bpfObject);

  // BPF program life cycle management
  void progAttach(progName);
  void progDetach(progName);

  // Allows retroactive policies (Section 4.3)
  void progExecute(progName);

  // BPF Map management
  // Get/change map parameters
  void mapSetSize(mapName, mapSize);
  int mapGetSize(mapName);
  map<string, int> getAllMapFds();

  // Control Garbage Connection via Pinning (Section 4.2)
  void progPin(programName, pinPath);
  void mapPin(mapName, pinPath);
  void unpin(pinPath);

  // Efficient shared map (Section 4.3)
  void setSharedMap(mapName, pinPath);
};
```

Listing 1: NetEdit's BPFAdapter Interface (Section 5.1).

NetEdit operates a local instance on each server. During startup, it examines the data-plane (i.e., the BPF ecosystem within the kernel), cleans up any residual state from previously running tuningFeatures (i.e., eBPF programs), and analyzes the configuration to determine the list of tuningFeatures to load. They are then attached to the data plane using a BPFAdapter abstraction, which is described in Section 5.

After initialization, the NetEdit control plane actively monitors changes in service deployment on the host by subscribing to the local instance of our container management system. When a change in service is detected, NetEdit identifies the tuning policies to activate (load and attach programs) or deactivate (unload and detach programs). To facilitate the management of specific tuningFeatures, NetEdit introduces the concept of a PolicyEngine, which coordinates the life-cycle and configuration of all programs required for a particular tuningFeature. The PolicyEngine utilizes the BPFAdapter interface to load or unload eBPF programs as needed. Once the programs are loaded, they intercept and inspect network traffic, determining when and how to apply the specified tuningFeatures based on the information stored in the ConfigStore.

5 NETEDIT DATA PLANE

NetEdit's data plane explicitly decouples the eBPF ecosystem from the kernel, allowing us to shift away from implicit reliance on the kernel for managing eBPF interactions to gaining explicit control, which eases the development of diverse use cases. We achieve this decoupling through several approaches discussed below – we have contributed several of our decoupling mechanisms back to the community (Section 9.3).

5.1 Decoupling Interface with BPFAdapter

BPFAdapter is an abstraction for standardizing the management operations, e.g., attach or configure, of eBPF programs across different hook-points by hiding the complexity associated with each attach-type. It provides such a uniform interface through which the control plane can manage the life-cycle of an arbitrary set of eBPF programs without understanding the nuanced hook-point-specific details associated with each eBPF program [2]. To this end, the interface supports the minimal common set of functionality required to manage the lifecycle of an eBPF program, i.e., initialization, configuration, deletion. We present the interface in Listing 1. It differs from opensource abstractions [11, 15] in two key ways:

Lazy Loading: To minimize CPU overheads associated with our tuningFeatures, our interface incorporates a *lazy loading* mechanism. Specifically, BPFAdapter delays the loading and attachment of any eBPF program until an active policy is present. This approach can save significant CPU and memory resources, especially for tuningFeatures that target specific services (We quantify these savings in Section 8.3.3). We can implement this lazy loading because our policy interfaces (Section 6.1) allow us to determine not only which services require a tuningFeature but also which traffic and classifier activate the tuningFeature when needed.

Explicit Storage Management: The limitations imposed by the kernel's default heuristics and implicit management hinder our ability to fully utilize eBPF programs and maps for diverse use cases. By striving for explicit life-cycle management of the entire eBPF ecosystem (*both maps and eBPF programs*), we can address these limitations, e.g., implementing map sharing across independent eBPF programs requires explicit lifetime management of maps to ensure correctness (discussed in Section 5.3). Existing abstractions [8, 11, 15] do not support explicit map management.

5.2 Decoupling Heuristics for High Availability

We achieve high availability of tuningFeatures with three mechanisms below.

Explicit Garbage Collection (*warm-reboots*) We introduce a primitive that allows NetEdit to control when the kernel performs eBPF-specific garbage collection. Traditionally, the kernel relies on reference counting to determine when to perform garbage collection. However, we leverage a recent eBPF capability called "pinning" to persist eBPF objects (programs or maps) by storing them in a local file system (pin-fs in Figure 2). This approach removes the dependency on reference counting for garbage collection.

During initialization, NetEdit explicitly performs cleanup, ensuring that eBPF objects remain available until NetEdit restarts. While the concept of an eBPF file system is used in several open-source frameworks, it is worth noting that: (i) these additions were made after our discussions at various industry conferences, and (ii) our approach considers both program and map objects, whereas existing frameworks primarily focus on programs alone, limiting their ability for ensuring correctness (shown in Section 5.3).

Retroactive Policies Enforcement (*bpf-iter*) It is, often, necessary to restart an eBPF program to update its code or configuration files. However, once attached, eBPF programs typically rely on kernel events to trigger their execution. For example, connection state-based eBPF programs (sockops) are invoked for all future connection state changes. This event-driven approach poses challenges for connections that start during the reboot period, as well as for long-lived connections that existed before the reboot. The problem arises because certain eBPF programs, which enforce policies during connection creation (attached to hook-points that are

[2]The BPFAdapter maintains a mapping of hookpoints to kernel-versions which allows it to determine which eBPF programs a host can support.

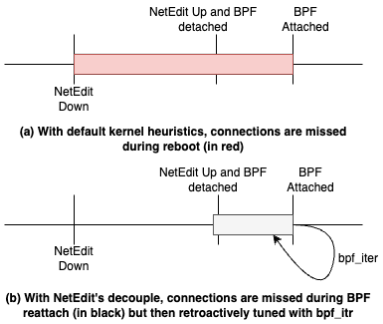

Figure 3: Comparison of NetEdit availability with default kernel heuristics (a) and NetEdit's heuristics (b).

only invoked during connection creation), will not be triggered for pre-existing connections since there is no specific event to register for a callback.

To address this issue, NetEdit offers a capability that allows the control plane to trigger an eBPF program and invoke the portion of the eBPF program responsible for tuning policies on all active connections thus allowing it to enforce its current policies. This capability assumes that eBPF programs are idempotent, meaning that applying a policy multiple times does not compromise correctness.

Fig. 3, shows how WarmReboots and bpf-iter allow NetEdit to provide highly available tuningFeatures.

Eliminate Data Fate-Sharing (*metrics exporter*) To ensure that the user space functionality required by the program persists even when NetEdit fails or reboots, we have taken steps to eliminate the dependency on the control plane for exporting debugging metrics and observability data. Instead, we have introduced a separate lightweight agent process dedicated solely to interacting with the eBPF programs and capturing/exporting measurement data to our data store. This approach compartmentalizes and isolates observability from the availability of the control plane, ensuring that data is consistently captured even during NetEdit failures or reboots. This approach differs from eBPF orchestrators [8, 11, 15] that bundle all functionality into a single userspace process.

5.3 Decoupling eBPF-to-eBPF for Correctness

The kernel provides inconsistent methods to facilitate interactions between eBPF programs, such as eBPF program chaining and sharing of maps. For instance, at certain attach points, the ordering of eBPF programs is determined by priority, while in others, it is based on the order of eBPF install/attach commands. This inconsistency implies that at our scale there can be a broad range of arbitrary eBPF program orderings across servers. [3] Additionally, composing eBPF programs can be challenging due to considerations of priorities and traffic directionality.

eBPF program composition: As discussed in Section 6.1, our policies are non-overlapping and are designed to be applied statically. This allows us to provide predictable functionality and ensure priorities by maintaining a strict and predefined ordering across the fleet based on predefined priorities. This ordering implicitly addresses conflicting policies by forcing higher priority eBPF programs to operate earlier while ensuring that DevOps can reason about the potential flow of traffic through NetEdit. This is distinct from existing opensource approaches that rely on the kernel, which provide unpredictable behavior because, as discussed earlier, the ordering of the eBPF programs is tied to the order of attachments. To address this, BPFAdapter maintains a globally predefined ordering of eBPF programs and we overlay an array over the hook-point. eBPF programs are loaded into predefined elements of the array according to their location in the globally defined ordering. Thus, it allows us to load (or reload) eBPF programs in an arbitrary order while still maintaining a strict attach/execution ordering. For eBPF programs at per-packet hook-points, i.e., *TC*, we have two arrays; one for ingress, and another for egress, which are mirror opposites of each other. This allows us to deal with the directionality requirement of per-packet eBPF programs.

SharedMaps: By default, when the kernel loads an eBPF program, it creates local state for each of its map structures which means when shared maps are used, without external coordination, the kernel may duplicate them instead of sharing them. BPFAdapter's explicit decoupling of state management from program management enables NetEdit to safely coordinate efficient sharing of state across programs. To ensure proper map sharing, BPFAdapter seamlessly provides the necessary coordination of eBPF program and map attach and load order. Specifically, BPFAdapter explicitly tracks different versions of shared maps and the tuningFeatures that explicitly register to share them. To ensure actual map sharing, NetEdit "pins" shared state to well-defined locations and loads eBPF programs in a manner that ensures the pre-defined shared state is used instead of creating local state. This approach guarantees that maps are shared as intended and avoids unnecessary duplication.

6 NETEDIT CONTROL PLANE

The NetEdit control plane performs two key functions: matching the configuration of desired tuningFeatures to services and flows, and enabling measurement instrumentation for visibility. In the following sections, we cover the policy configuration specification (Section 6.1), context-specific orchestration (Section 6.2), initialization process (Section 6.3), and monitoring and auditing (Section 6.4).

6.1 Policy Store

The "policy store" contains the policies that selectively enable each tuningFeature. Each policy contains the following three components; we provide a sample policy for our DCTCP tuningFeature [16] in Listing 2.

The **Classifier** component represents the local and remote endpoints to which a policy will be applied. In our environment, the local endpoint is typically defined as a set of jobs, while the remote endpoint is specified based on relative distance (such as same-rack, same-datacenter, or same-continent) for the traffic. This primitive enables us to collaborate with job owners and control whether tuningFeatures are enabled across the entire backbone or limited

[3]The set of attached eBPF programs is based on the set of active policies. Moreover, we lazy-attach programs for efficiency reasons. Thus, for both reasons, we can expect arbitrary orderings or an arbitrary set of programs.

```
TcpCcaPolicy(
            info=TrafficInfo(name="bpf_dctcp"),
            classifier=TrafficClassifier(
                source=Matchers(
                   Tiers=["storage"],
                   localities=["USEast", "USWest"],
                ),
            ),
            config=TcpCcaConfig(
                ccaName="bpf_dctcp",
                tcpParam=DctcpParameters(
                    dctcp_shift_g(inRack=2, inDc=3, inRegion=4), /* g = 1/2^4 */
                    dctcp_alpha_on_init=1024,
                ),
            ),
            disablelist=["host12","host323"],
        ),
```

Listing 2: Example Policy Config for the DCTCP tuningFeature [16].

to specific datacenters. However, NetEdit needs to maintain accurate information about the executing jobs and the assignment of addresses to different locations (as discussed in Section 6.2).

The **Config** component includes fields that specify the tuningFeature's arguments and parameters. These are represented as a key-value store which allows different tuningFeatures to have an arbitrary number and type of parameters.

The **Disable list** is a matcher that applies to the local "source" endpoint and allows the complete disabling of a policy. This property has proven useful when debugging issues, as it allows us to quickly rule out policies that are not relevant to the problem at hand. By applying this change as an exception to the rule, rather than modifying the rule itself, it becomes easier to undo and provides more flexibility in crafting narrow exceptions.

6.2 PolicyEngine

The PolicyEngine evaluates each configuration policy to determine whether to load the eBPF programs associated with a tuningFeature and populate a BPF map with the appropriate configuration [4]. In our design, we prioritize stability and debuggability over optimality. Therefore, our policies are designed to be applied statically, leveraging the structured allocation of addresses in our topology. For stability, we did not react to measured latency or performance characteristics. Specifically, we assign addresses hierarchically to our global infrastructure. For example, addresses for clusters in a data center are grouped within a specific prefix, and addresses for racks (sets of physically identical servers) are assigned subprefixes. This hierarchical assignment of IP addresses provides a concise and compressible method for inferring approximate locality and, to some extent, hardware characteristics.

The PolicyEngine consists of two components that bridge the gap between low-level traffic attributes (such as addresses) and high-level policy specifications (such as jobs, locality, and hardware). The first component, called ScopeResolver, utilizes a machine-readable network map to map address prefixes to relative distance scopes, such as rack, datacenter, or continent. This compact representation of relative distance is useful for specifying policy, similar to its use in selecting nearby replicas. The second component, InfraResolver,

[4]The BPF map is shared in-memory state comprising global variables (e.g., a debug flag) or lookup structures (e.g., a radix trie for address prefix mapping).

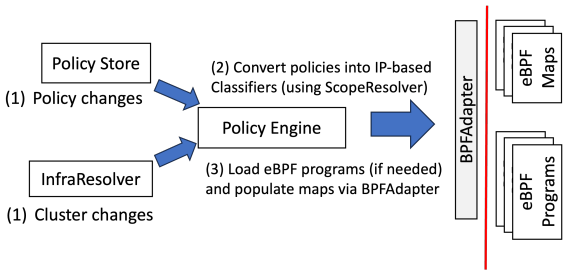

Figure 4: NetEdit's PolicyEngine Loop.

queries the container orchestrator (Twine Agent) to determine active services and the addresses associated with those services. This dynamic capability sets NetEdit apart from other eBPF managers like cilium, as it allows for the dynamic loading of eBPF programs into the Kernel based on the active services, rather than statically loading them during service deployment.

The PolicyEngine operates in a loop, shown in Fig 4, continuously updating the network map and checking the set of locally running services to determine whether policies should be loaded, unloaded, or reconfigured.

6.3 The Initializer

The Initializer serves three key purposes: (a) It ensures that tuningFeatures are applied in a deterministic order, (b) It satisfies dependencies between tuningFeatures, and (c) It applies policies to connections in progress during various system states, such as graceful restarts, restarts after a crash, or initial startup. When multiple eBPF programs attach to the same operation, such as intercepting setsockopt(), it is possible for the system's behavior to depend on which program processes the setsockopt call first. To ensure deterministic execution, we attach programs in a predefined order. This helps maintain consistent behavior and mitigates potential bugs that may arise from the order of program execution.

During startup, the Initializer performs several tasks to ensure the smooth operation (Fig. 5). First, it cleans up previous instances of NetEdit by detaching and unpinning programs. Next, it loads new tuningFeatures and pins them to ensure their availability. The complexity of pinning programs allows the BPF code to function independently when the user-level component of NetEdit restarts. This means that new flows appearing while NetEdit is restarting can still be properly classified. For certain policies, the eBPF programs can be applied to live connections using *bpf-iter*, allowing them to take effect immediately. By ensuring the operational status of tuningFeatures despite code restarts, the monitoring components, which will be described next, are simplified. There is no need for exceptions for connections that started while NetEdit was restarting or that existed before loaded.

6.4 MetricExporter and Auditor

We employ two methods to monitor the operation of tuningFeatures: the MetricExporter and the Auditor.

The **MetricExporter** collects counters that are incremented by eBPF programs. These counters track control flow and include counts for successful operations (e.g., marked packets, parameter

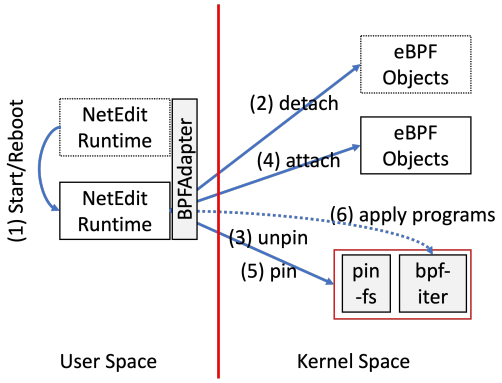

Figure 5: Workflow for Initialization of NetEdit.

	Tuning Feature	Description	Hookpoint	Granularity
1	Init CWND	tune initial window	Sockops	Transport
2	RWND	tune receiver window (flow control)	TC	
3	Connection timeout	Tune initial connection timeout	Sockops	
4	Pacing Tuner	Tune flow pacing rate	Sockops	
5	MSS Tuner	Toggle Jumbo MTU setting	Sockops	
6	BPF DCTCP	New Congestion Control	Struct_ops Sockops	
7	D2TCP	New Congestion Control	Struct_ops Sockops	
8	BET	New Congestion Control	Struct_ops Sockops Sockopt	
9	ILA	V6 virtual networking	TC	Connection
10	Nat64	Nat64 gateway	TC	

Table 3: TuningFeatures developed on NetEdit.

changes) as well as counts for failed operations (e.g., processing errors for malformed packets). Each tuningFeature has its own set of counters, and user-level logic is required to interpret the BPF map containing the counter array. To ensure continuous processing of these counters, even during NetEdit restarts, we separate the MetricExporter process from the rest of NetEdit. However, we need to inform the MetricExporter about which tuningFeatures have counters that need to be collected and exported.

The **Auditor** verifies the behavior of eBPF programs over a longer timescale. It leverages FBFlow [47] to sample a small fraction of outgoing packets and adds additional data about socket state and the services involved at both endpoints. To ensure accurate confirmation of correct tuningFeature operation, we include relevant fields in the collected socket state. For example, we collect receive window statistics and connection MSS limits, which were not available for flow monitoring. Recording these fields within FBFlow's BPF is straightforward and does not significantly increase the data size, as the fields are typically small integers with low entropy that can be easily compressed. This system allows us to verify that a tuningFeature is correctly applied to the specified jobs and connections within the intended scope.

The MetricExporter and Auditor components work together in a complementary manner. The MetricExporter confirms that a tuningFeature executes without errors, while the Auditor ensures that it effectively transforms traffic.

7 APPLICATIONS

NetEdit enables the rapid development and deployment of a wide range of tuningFeatures, resulting in significant network improvements.

7.1 TuningFeature Diversity

Over the past five years, we have developed more than **10** tuningFeatures (see Table 3). These tuningFeatures are diverse in terms of their specific functionality, the problems they solve, their implementation, and the level of granularity at which they operate. From Table 3, we highlight two key observations.

First, all but two of our tuningFeatures operate at the transport level, which is not surprising given prior work[2] that has shown significant performance gains in modern data centers at the transport level. Even within the transport layer, there is a diversity in the hook-points at which individual tuningFeatures operate, attaching at Struct_ops, SockOps, or TC hook-points, reflecting the diversity of the tuningFeatures: specifically, they either dynamically set specific configuration parameters for existing kernel congestion control implementations (rows 1-5) or enable congestion control algorithms written entirely in eBPF, providing more parameter tuning flexibility (rows 6-8). We note that we have chosen to implement experimental congestion control variants or reimplement a subset of existing variants (DCTCP[2]) in eBPF due to its benefits in terms of experimentation, testing, and rollout - essentially allowing us to bypass the entire kernel stack for the end-to-end applications, saving months of time and manpower per tuningFeature.

Second, the eBPF congestion control implementation tuningFeatures (rows 6-8) operate at multiple hook-points. This is expected as a congestion control algorithm is complex and requires coordination and coupling across various aspects of a connection, which are exposed by different hook-points. NetEdit simplifies this entire process, as we will explain next.

7.2 TuningFeature Building Blocks

This diversity is possible due to the comprehensive capabilities provided by NetEdit, which simplifies the development, testing, and deployment of these tuningFeatures.

We note that the foundation of NetEdit's flexibility and power lies in the reuse of NetEdit's capabilities across tuningFeatures. In Table 4, we list the capabilities and the number of tuningFeatures that use them. Generally, the capabilities can be categorized into three classes.

First, there are the "core" capabilities, a small group of capabilities that are used by *all* tuningFeatures. As expected, these capabilities are the core abstractions and functionalities provided by NetEdit. They consist of the abstraction interface (BPFAdapter (Section 5.1)), MetricsExporter (Section 6.4), the continuous deployment pipeline (i.e., Prod A/B testing, and Static Configs (Section 6.3). Recall, every tuningFeature that uses the eBPF API is thoroughly tested and easily configurable in production.

Second, there are the "transport" capabilities, which encompass common functions necessary for tuning various aspects of the transport protocol. For example, the ScopeResolver enables connection classification based on round-trip time (RTT) and location. These capabilities are specifically utilized by transport tuningFeatures.

Type	Release	Capability	Description	# of fea.
Data Plane	H1 2022	BPFAdapter (Section 5.1)	Lifecycle Management Interface	10
	H1 2022	Shared Maps (Section 5.3)	Sharing state across tuningFeatures	3
	H2 2022	bpf-iter (Section 5.2)	apply eBPF programs to existing connections	1
	H1 2023	Warm-Reboot (Section 5.2)	Always on BPF dataplane	2
Control Plane	H2 2021	PolicyEngine (Section 6.1)	Dynamical policy engine	7
	H1 2020	Static Config. (Section 6.3)	Static policy engine	10
	H1 2021	ScopeResolver (Section 6.1)	Infers connection locality	6
Debug	H2 2021	Diagnosis (Section 6.4)	Debug Counters	10
	H1 2022	Prod A/B Testing (Section 9.1)	Prod canary for all tuningFeatures	10

Table 4: Capabilities provided by NetEdit.

Finally, there are the "emerging" capabilities, which encompass a set of newer functionalities that enable tuningFeatures to share state (SharedMaps Section 5.3) or facilitate fast and reliable rollouts of the NetEdit binary (WarmReboots Section 5.2). Currently, these capabilities are utilized by a small number of tuningFeatures, but we are in the process of retrofitting older tuningFeatures that could benefit from these capabilities.

7.3 Developer Productivity Improvement

The modularized capabilities of NetEdit have resulted in significant improvements in tuningFeature development time, effort, and reliability. We perform a quantitative comparison of the development and deployment times for three tuningFeatures, focusing specifically on transport tuningFeatures to ensure a fair comparison. We observe a substantial decrease in development time, from approximately 6 months to just a few weeks. This drastic reduction in development and deployment time can be attributed to the modular nature of NetEdit's capabilities. Firstly, the introduction of rich yet simple abstractions and runtime frameworks provided by NetEdit's interface enables faster development of the core feature. Secondly, the reuse of common utilities such as connection scope resolvers further reduces development time and allows developers to focus on the core business logic. Lastly, the automated testing and rollout mechanisms ensures higher confidence and faster deployment.

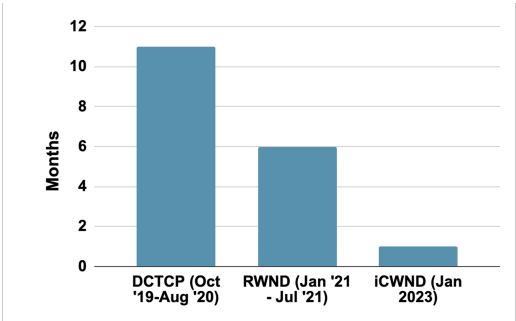

Figure 6: TuningFeature Development Speed.

To quantify the productivity impact of NetEdit in terms of code changes, we measured the extent of code sharing. We focused on a more recent feature, PacingTuner (see Table 3), and examined the number and type of lines of code added for this feature. The core eBPF logic accounted for approximately 150 lines, while the business logic (feature code) consisted of around 500 lines. This amounts to less than 1.5% of the total lines of code in NetEdit. Additionally, we analyzed the aggregate number of commits across NetEdit (see Table 5). Over a six-month period, the majority of changes (40.08%) were related to policy, followed by BPF (24.62%).

	Data Plane	Control Plane			Configurations
	(BPF)	Runtime	Monitoring	Policy Engine	Policy Store
%	24.62%	13.4%	11.64%	16.6%	40.08%

Table 5: Breakdown of code commits (Jan-July 2023).

7.4 Design Lessons For Hookpoint Selection

While NetEdit simplifies the development of tuningFeatures, the selection of an eBPF program's hook-point requires careful consideration to strike a balance between control and CPU overhead; minimizing CPU overhead is our primary goal. As our services have evolved, we often find ourselves rewriting our tuningFeatures to work at other hook-points. Next, we discuss the process of hook-point selection and summarize the design lessons we have learned.

Benefits of Hookpoint Selection: In the case of BET-tuner, which offers a scavenger congestion control implementation, a service can choose to deprioritize traffic by using BET through a setsockopt option. Initially, BET-tuner utilized the sockops hook-point and relied on RTT callbacks to detect changes in Quality of Service (QoS) for the socket. However, by switching to a proactive "socksetopts"-based approach, we were able to significantly reduce the number of calls required to detect changes. With this approach, our eBPF program is only triggered when any socket option is modified, resulting in a substantial decrease in BPF calls and execution for each connection, sometimes by up to 200 times.

Dangers of Hookpoint Selection: In the case of the RWND-feature (Table 3), our initial design operated at the TC level, rewriting the RWND (Receive Window) of every packet. However, due to concerns about the per-packet overhead, we considered shifting towards the sockops hook-point, which operates at the connection level. However, we ultimately decided to revert to the more expensive approach because we discovered that the Kernel provided different default behavior at the higher hook-point. Implementing the desired behavior would have required invasive changes to the Kernel's core networking stack.

Lessons: Hook-point selection needs careful consideration of several factors. First, it is important to ensure that it satisfies the correctness requirements and provides the necessary functionality for the feature. Second, it is crucial to characterize the associated performance overheads through extensive stress tests and appropriate monitoring mechanisms. Lastly, it is essential to have a deep understanding of the functionality within the Kernel that eBPF interfaces may depend on. While the documentation is typically clear about the first aspect and extensive observability tools exist for the second aspect, the third aspect requires a broader community effort. This includes creating richer documentation with complex code

examples and developing semantic testing tools that can detect functionality changes as the eBPF ecosystem evolves. Such efforts will contribute to a better understanding of the Kernel's internal functionality and facilitate more informed hook-point selection decisions.

8 EVALUATION

Next, we evaluate and analyze our deployment of NetEdit.

8.1 Deployment Characterization

First, we analyze the deployment of NetEdit across our infrastructure. On average, we observe that approximately 98.4% or more of the servers in each of our data centers have NetEdit installed. It is important to note that NetEdit is not deployed on 100% of our fleet due to certain kernel fixes that impose a version restriction. [5]

Next, we examine the deployments of tuningFeatures and observe significant diversity. This diversity is naturally in alignment with the varied and heterogeneous needs of our services and hardware. We observe that the usage of features clusters into three groups: The first consists of tuningFeatures that are used by all NetEdit-enabled servers. These tuningFeatures mainly include transport features such as InitCWND and BET. The second consists of a single special tuningFeature called Touchstone. We use Touchstone to canary new NetEdit primitives, and therefore it is only enabled on a small fraction (10%) of our fleet at any given time. The last comprises niche tuningFeatures such as MSS Tuner, which is deployed on less than 2% of the fleet. These features are designed for specific services. Figures are not included due to space limit.

8.2 Performance Benefits of tuningFeatures

We quantified the performance benefits of customization and tuningFeatures enabled by NetEdit through two approaches. Firstly, we reported the performance benefits of the individual tuningFeatures when initially deployed. Secondly, we measured the performance degradations to the network and services when NetEdit was disabled. To disable NetEdit, we turned it off in an entire region comprising several data centers for a duration of 1 hour and 15 minutes. This disabled all tuningFeatures in that region and forced the services to utilize the default networking stack.

8.2.1 Performance Benefits of NetEdit Features. We have deployed several NetEdit features across multiple services over the years, resulting in significant performance benefits and wins. A selection of these benefits are summarized in Table 6. We observed notable improvements in tail latencies for various services, ranging from storage to Ads, using transport tuningFeatures such as congestion control algorithms, window sizes, and packet sizes. Additionally, these tuningFeatures allowed us to utilize our infrastructure more efficiently, doubling the load on certain storage servers without being limited by network bursts, and substantially saving cost by downgrading low-priority, cross-region transfers. Furthermore, transport tuning has led to a significant reduction in network retransmits, resulting in improved resource utilization and enhanced troubleshooting capabilities due to the reduced noise in our alarms.

[5] A small percentage of our fleet runs on older kernel versions, which prevents the installation of NetEdit.

Tuning Feature	Infra Win	Application Win
Init CWND		30% faster image generation 30% drop in cross-region transfer P99 latency for Storage app
RWND	90% fewer retransmits for storage app Taming bursts eliminated network bottleneck, allowing archival app to double load on servers	100% faster storage reads 67% reduction in P99 latency for Storage app
Pacing Tuner		40% improvement in P99 read latency for Storage
MSS Tuner	15% CPU savings for Storage, 6% for Ads	3.5% more throughput for Storage, 6% lower network utilization for Ads
DCTCP	40% improvement in P99 latency for commerce app 40% reduction in P99 latency for search app	75% reduction in network retransmits
BET		Significant network opex savings due to throttled traffic

Table 6: Performance wins due to TuningFeatures.

8.2.2 Performance Degradation upon Disabling NetEdit. When analyzing the impact of disabling NetEdit's tuningFeatures, we focus on three key layers: the services, the host, and the network. At the service layer, we examine service-specific performance metrics for the top two services in the region: Storage and MLInference. For MLInference, we observe an average 29% drop in queries per second, with a significant 59.21% drop in the 99th percentile. For the Storage service, we observe an average 37.14% drop in block read latency, with a substantial 65.85% drop in the 99th percentile. To understand the drop in service-level performance, we analyze host-level performance metrics, specifically focusing on transport-level metrics. In general, we observe a 4.5x increase in all retransmissions and a concerning 29x increase in retransmissions triggered by timeouts. Increased congestion at the top-of-rack (ToR) switch is identified as the cause of the service-level performance degradation. Additionally, we observe a 10% drop in throughput for both the uplink and downlink. At the network level, there is a significant 363.7% increase in ToR switch downlink congestion discards. In essence, the congestion and resulting application-level degradation are caused by heightened incast-style traffic patterns from applications, which are mitigated by the various tuning mechanisms deployed through NetEdit.

8.3 Resource Overheads

We evaluate the overheads of NetEdit along these dimensions:

8.3.1 Control Plane: We focus on the resource overheads of NetEdit by analyzing the userspace CPU and memory usage. Across our fleet of servers, we observe that NetEdit utilizes an average of 0.00023% (0.00026% at the 95th percentile) of CPU and 26MB (60.5MB at the 95th percentile) of memory. These low overheads are primarily due to the minimal functionality provided by NetEdit to support life-cycle management.

8.3.2 Data Plane: We observe that the eBPF programs in the data plane have a lightweight footprint and consume minimal resources. While we do not show the specific figures here due to space limitations, it is important to note that this minimalistic footprint is intentional, as larger footprints would be unacceptable in a large-scale production setting like ours. Several capabilities, such as shared

maps (Section 5.3) and lazy loading (Section 5.1), allow us to explicitly reduce memory or CPU footprint by reusing Maps, avoiding duplicate functionality, and deferring the loading until matching policies exist.

8.3.3 Lazy Loading Cost Savings. This section analyzes the performance benefits of Lazy loading (Section 5.1) This component of NetEdit allows us to ensure that only the required eBPF programs are attached, thus saving resources. In comparing two similar tuningFeatures, one that uses lazy loading and one which does not, we observed that lazy loaded eBPF programs consume significantly less CPU across 10 data centers – on aggregate, they consume on avg 14% (and 27% at p99) of the CPU consumed by the non-lazy loaded eBPF programs.

9 EXPERIENCES, OUTAGES, AND OPEN SOURCE

In this section, we present a comprehensive overview of our experiences over the past five years. We discuss the challenges and failures we encountered and the efforts we invested in addressing them. We summarize our contributions to the community.

9.1 Deployment Experiences

Although the eBPF verifier ensures that an eBPF program is safe and will not crash the kernel, we have found that eBPF programs can still trigger bugs in the underlying kernel interfaces, leading to large-scale production outages. These bugs cannot be detected by testing eBPF programs alone in isolation. We first share our experiences of integrating eBPF into our testing microservices framework below.

We attempted to apply our standard testing methodology, which includes unit tests and production A/B tests for microservices, to our eBPF programs ecosystem. However, we encountered challenges due to the strong coupling between eBPF and the kernel, which hindered our ability to effectively reuse existing testing methodologies. To address these challenges, we implemented the following approach.

First, our unit tests ensure that the eBPF program loads correctly, simulates packets or connections (via test-cgroup) using packetdrill [13], and validates eBPF behavior by analyzing eBPF program measurement such as counters and connection parameters. Second, We run unit tests in production environment alongside diverse production workloads. While it is relatively straightforward to isolate multiple instances of the same microservice due to existing virtualization primitives like cgroups, it does not work for eBPF. At certain layers of the kernel, the eBPF-kernel does not support isolation and virtualization of multiple instances of the same eBPF program. For example, at the BPP-TCP-CCA attach point, it is not possible to support multiple eBPF programs with the same name. This is a unique challenge that arises due to the broad range of hook-points that we support. Other frameworks, such as BPFMan/BPFD[11] or Leaf[54], which focus on XDP and TC, do not have these issues.

To enable parallel testing and isolation of test eBPF programs from each other and production eBPF programs, we introduced a set of mechanisms that dynamically alter test eBPF programs before deploying them. These include using dummy cgroups to isolate test and production eBPF programs on the same host, and for hook-points where isolation is not possible, such as `struct_ops`, we rewrite test eBPF program names to avoid collisions with production versions when loaded. Note that no additional changes are required to use our A/B-testing infrastructure for NetEdit [52].

Classification of Production Bugs (by root cause)	%
Bugs in NetEdit platform (configuration or code)	27%
Bugs in Kernel	18%
Operational Issues	55%

Table 7: Characterization of Production Bugs/Outages.

9.2 Production Outages

9.2.1 Failure Classification. Table 7 characterizes the outages related to NetEdit over the last five years.

Kernel/Infra Bugs Our extensive use of the eBPF-kernel interface in a heterogeneous environment has uncovered numerous previously undetected bugs in the kernel. We have collaborated with the community to address many of these issues. The bugs can be categorized into three classes: those related to the kernel-BPF interface, eBPF-to-eBPF composition interface, and kernel-version specific bugs.

- **Class #1: Kernel-eBPF Interface Bug:** The diversity of our use cases forces our eBPF programs to interact with the kernel in often unexpected ways that can trigger latent kernel bugs. For example, recursive calls to an eBPF function (e.g., ssthresh), caused the kernel to ignore its return value and read uninitialized register with random values thus triggering service unavailable due to out of memory errors. [6].
- **Class #2: Kernel Portability Bugs:** Different kernel versions provide different levels of functionality for the kernel-eBPF interfaces. As such, the behavior of our tuningFeatures can change radically between versions. For example, re-attaching tuningFeatures (i.e., TC BPF) on some kernels but not others triggered a kernel Panic. [7] This bug is particularly interesting because it is triggered during subsequent attempts to reattach the tuningFeature– a very rare event.
- **Class #3: Cross TuningFeature Interactions:** Traditionally, eBPF programs interact with each other via shared maps. We, however, observed indirect interactions via the mechanisms provided for attaching multiple eBPF programs to a hook-point – a failure or bug in one eBPF program could prevent other eBPF programs from executing (specifically, such failures could prevent the kernel from calling other eBPF programs at the same hookpoint). Such bugs can be challenging because the exact combination of features and configurations can be large.

Other Infra Dependencies The largest class of bugs and outages are caused by interactions between NetEdit and the underlying infrastructure, service workloads (i.e., traffic), or eBPF programs not managed by NetEdit (e.g., container virtualization eBPF programs).

[6] As a result of these random values, one of our tuningFeatures was setting arbitrary ssthresh values for connections which resulted in arbitrary congestion windows and thus a service outage. Specifically, when the arbitrary value led to a low congestion window, the connections would take a longer time to finish which led to an increase in the number of connections and subsequently timeout errors and OOM errors.

[7] Specifically, this occurred on older kernels because a synchronization primitive (RCU grace period) was not respected for TC hookpoint's RCU pointer.

NetEdit and its features operate under certain assumptions about the environment, and unexpected changes can lead to failures. For example, due to an error in our infrastructure automation scripts, the eBPF file system for pinned objects (i.e., /sys/fs/bpf) was double mounted, causing NetEdit's initializer to be unable to unpin the old objects while trying to attach new ones because the old objects were still attached (hidden under the old mount). In such scenarios, debugging and diagnosis can be particularly challenging as the problem may only manifest on a small number of hosts.

9.2.2 Lessons. In response to some of the outages previously discussed, we have made several changes to our eBPF testing framework that differentiate it from our traditional CI/CD pipeline for microservices deployment. We have introduced an integration testing phase between unit tests and A/B testing that focuses on testing tuningFeature-to-tuningFeature interactions and detecting "Cross Tuning Feature Interactions" bugs. During these tests, we deploy multiple features and use shadow production traffic to test them.

Although we have enhanced our pipeline to more thoroughly detect bugs, we still occasionally discover them in production. We highlight open research directions that can better assist us and the eBPF community: (i) there is a need for rigorous testing frameworks, such as fuzzing-based, to expose hidden bugs in the eBPF-kernel interfaces. Unlike prior work on kernel testing, we observed that due to eBPF's coupling, our eBPF programs either explored uncommon kernel paths or introduced new interactions that did not exist before. (ii) we observed that in addition to testing the eBPF-kernel interfaces, we also require frameworks that analyze the interactions between different combinations of features. However, these testing frameworks crucially need dynamic instrumentation tools for automatic logging to capture sufficient information for tuningFeature validation.

9.3 Opensource and Community Engagement

So far, we have upstreamed several components of NetEdit's data plane into the kernel and are working to open source NetEdit's adapter. To support NetEdit's diverse requirements, we have successfully upstreamed kernel changes as follows:

Exposing more functionality: We introduced struct_ops[30], an interface for implementing TCP congestion control algorithms (CCA) in eBPF through specific kernel function pointers. We further extended the kernel to allow struct_ops programs to: (i) override or tune a subset of kernel behavior by adding the kfunc interface[32] and (ii) be atomically upgraded by adding bpf_link [35]. Crucially, treating CCA as a feature provides flexibility for iterative TCP CCA development without requiring kernel bypass or being blocked by kernel rollout timelines.[8]

Improve kernel primitives: To enable NetEdit's programs to access the same rich functionality as user-space applications without burdening the kernel with duplicated code, we refactored the "bpf_setsockopt()" helper functions to allow NetEdit eBPF programs to have feature parity with the kernel networking stack's "setsockopt()"[33]. To allow NetEdit to retroactively apply policies (Section 5.2), we added bpf_setsockopt support to bpf-tcp-iter[31] which provided a novel out-of-band mechanism for updating TCP parameters.

[8]We also have the ability to safely fall back to kernel defaults if required.

10 RELATED WORK

eBPF Related Work: Contemporary eBPF-orchestration frameworks [8, 11, 15] do not address many of NetEdit's key requirements, e.g., the heuristic (Section 5.2) and BPF-to-BPF decoupling (Section 5.3) or Integrated Lifecycle Management: (Section 6). We elaborate on them in Appendix 3. There is extensive amount of work on improving eBPF ecosystem: enhancing the verifier [4, 9, 18, 24, 26, 29, 42, 43, 53, 55], demonstrating applications of eBPF [4, 10, 20, 27, 45, 57], introducing new primitives [3, 12, 22, 28, 56]. NetEdit explores an orthogonal space by providing a suite of abstractions, designs, frameworks to enable high-velocity eBPF code changes in a highly dynamic environment. Although NetEdit focuses on networking, we anticipate that several key concepts are easily translatable to the storage and compute domains.

EndHost Network Tuning Techniques. At Meta, we chose eBPF over the broad set of techniques for tuning the Linux kernel network stack [1, 41, 44] (e.g., eBPF programs, kernel modules, SetSockOpts, IPTables) for reasons discussed in Section 2.1 and Appendix A.

SDN Related Work: Although philosophically inspired by SDN's traditional decoupling of functionalities [25, 38, 46, 48], NetEdit fundamentally advances in two ways: first, it introduces design choices to decouple kernel resource management policies from eBPF, second, it provides one of the first realizations of SDN wherein centralized state is not required for effectively calculating optimal policies.

11 CONCLUSION

Identifying a balance point in network tuning is particularly challenging when operating with services and device heterogeneity at data center scale. NetEdit is a platform that provides network tuning as a service. It manages both host network stack defaults and customized parameters for services based on their latency, throughput, and burst profiles. NetEdit's decoupling of policy configuration specification from enforcement allows customization without tight coupling with services or making physical network assumptions. While NetEdit focuses on introducing design choices, principles, and interfaces within the context of network tuning, the decoupling of eBPF management (development, deployment, and testing) from kernel's default behavior that NetEdit introduces transcends the networking domain. We anticipate that these principles are useful to the more recent eBPF storage and scheduling domains.

Ethical Considerations: This work does not raise any ethical issues.

12 ACKNOWLEDGMENTS.

This paper presents the work of several teams at Meta who helped realize this project. We thank Andrey Larin, Anshima Gupta, Hechao Li, Manikandan Somasundaram, Nathan Walker, Omar Baldonado, Petr Lapukhov, Satendra Gera, Shishir Bharadwaj, Sharon Li, Miao Xu and Takshak Chahande. We would also like to thank our reviewers and shepherd K.K. Ramakrishnan for their insightful feedback.

REFERENCES

[1] Mohammad Al-Fares, Khaled Elmeleegy, Benjamin Reed, and Igor Gashinsky. Overclocking the yahoo! cdn for faster web page loads. In *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference*, IMC '11, page 569–584, New York, NY, USA, 2011. Association for Computing Machinery.

[2] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center tcp (dctcp). *SIGCOMM Comput. Commun. Rev.*, 40(4):63–74, aug 2010.

[3] Nadav Amit and Michael Wei. The design and implementation of hyperupcalls. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 97–112, 2018.

[4] Ido Ben-Yair, Pavel Rogovoy, and Nezer Zaidenberg. AI & EBPF based performance anomaly detection system. In *Proceedings of the 12th ACM International Conference on Systems and Storage*, SYSTOR '19, page 180, New York, NY, USA, 2019. Association for Computing Machinery.

[5] Theophilus Benson, Palanivel Kodeswaran, and Sayandeep Sen. OPENED tool for managing eBPF heterogeneity. In *Linux Plumbers Conference*, 2022.

[6] Mauricio Vásquez Bernal and Rafael David Tinoco. BTFGen: One step closer to truly portable eBPF programs. In *eBPF Summit*, 2022.

[7] Gilberto Bertin. Axdp in practice: integrating xdp in our ddos mitigation pipeline, 2017.

[8] Gilberto Bertin. Cilium CNI: eBPF-based Networking, Observability, Security, 2017.

[9] Sanjit Bhat and Hovav Shacham. Formal verification of the linux kernel eBPF verifier range analysis, 2022.

[10] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. Theseus: an experiment in operating system structure and state management. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–19, 2020.

[11] bpfman: An eBPF Manager. https://bpfman.io/main/, 2024.

[12] Lawrence Brakmo. TCP-BPF: programmatically tuning TCP behavior through BPF. In *Netdev 2.2, The Technical Conference on Linux Networking*, 2017.

[13] Neal Cardwell, Yuchung Cheng, Lawrence Brakmo, Matt Mathis, Barath Raghavan, Nandita Dukkipati, Hsiao keng Jerry Chu, Andreas Terzis, and Tom Herbert. packetdrill: Scriptable network stack testing, from sockets to packets. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 213–218, San Jose, CA, June 2013. USENIX Association.

[14] Cloudflare. Unimog - Cloudflare's edge load balancer, 2020.

[15] DataDog. eBPF manager. This manager helps handle the life cycle of your eBPF programs. https://github.com/DataDog/ebpf-manager, 2023.

[16] Abhishek Dhamija, Balasubramanian Madhavan, Hechao Li, Jie Meng, Shrikrishna Khare, Madhavi Rao, Lawrence Brakmo, Neil Spring, Prashanth Kannan, Srikanth Sundaresan, and Soudeh Ghorbani. A large-scale deployment of DCTCP. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 239–252, Santa Clara, CA, April 2024. USENIX Association.

[17] Facebook. A high performance layer 4 load-balancer, 2018.

[18] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. Simple and precise static analysis of untrusted linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, page 1069–1084, New York, NY, USA, 2019. Association for Computing Machinery.

[19] Ehab Ghabashneh, Yimeng Zhao, Cristian Lumezanu, Neil Spring, Srikanth Sundaresan, and Sanjay Rao. A microscopic view of bursts, buffer contention, and loss in data centers. In *Proceedings of the 22nd ACM Internet Measurement Conference*, IMC '22, page 567–580, New York, NY, USA, 2022. Association for Computing Machinery.

[20] Yoann Ghigoff, Julien Sopena, Kahina Lazri, Antoine Blin, and Gilles Muller. BMC: Accelerating memcached using safe in-kernel caching and pre-stack processing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 487–501, 2021.

[21] Thomas Graf. How eBPF will solve service mesh - goodbye sidecars, December 2021.

[22] Brendan D Gregg. Linux enhanced BPF (eBPF) tracing tools. *URL: http://www. brendangregg. com/ebpf. html*, 2023.

[23] IO Visor. eBPF: extended Berkeley Packet Filter. https://www.iovisor.org/technology/ebpf, 2016.

[24] Mathieu Jadin, Quentin De Coninck, Louis Navarre, Michael Schapira, and Olivier Bonaventure. Leveraging eBPF to make TCP path-aware. *IEEE Transactions on Network and Service Management*, 19(3):2827–2838, 2022.

[25] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, et al. B4: Experience with a globally-deployed software defined wan. *ACM SIGCOMM Computer Communication Review*, 43(4):3–14, 2013.

[26] Jinghao Jia, Raj Sahu, Adam Oswald, Dan Williams, Michael V. Le, and Tianyin Xu. Kernel extension verification is untenable. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, HOTOS '23, page 150–157, New York, NY, USA, 2023. Association for Computing Machinery.

[27] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. Programmable system call security with eBPF. *arXiv preprint arXiv:2302.10366*, 2023.

[28] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. Syrup: User-defined scheduling across the stack. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 605–620, 2021.

[29] Hsuan-Chi Kuo, Kai-Hsun Chen, Yicheng Lu, Dan Williams, Sibin Mohan, and Tianyin Xu. Verified programs can party: Optimizing kernel extensions via post-verification merging. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 283–299, New York, NY, USA, 2022. Association for Computing Machinery.

[30] Martin KaFai Lau. Introduce BPF STRUCT_OPS. https://lore.kernel.org/bpf/20200109003453.3854769-1-kafai@fb.com/, 2020.

[31] Martin KaFai Lau. bpf: Allow bpf tcp iter to do bpf_(get|set)sockopt. https://lore.kernel.org/bpf/20210701200535.1033513-1-kafai@fb.com/, 2021.

[32] Martin KaFai Lau. bpf: Support calling kernel function. https://lore.kernel.org/bpf/20210325015124.1543397-1-kafai@fb.com/, 2021.

[33] Martin KaFai Lau. bpf: net: Remove duplicated code from bpf_getsockopt(). https://lore.kernel.org/all/20220902002750.2887415-1-kafai@fb.com/, 2022.

[34] Martin KaFai Lau. bpf: Remove recursion check for struct_ops prog. https://lore.kernel.org/bpf/20220922225616.3054840-1-kafai@fb.com/, 2022.

[35] Martin KaFai Lau. Transit between BPF TCP congestion controls. https://lore.kernel.org/all/20230323032405.3735486-1-kuifeng@meta.com/, 2023.

[36] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: A microkernel approach to host networking. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 399–413, New York, NY, USA, 2019. Association for Computing Machinery.

[37] Matt Fleming. A thorough introduction to eBPF. https://lwn.net/Articles/740157/, 2023.

[38] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM computer communication review*, 38(2):69–74, 2008.

[39] Sebastiano Miano, Fulvio Risso, Mauricio Vásquez Bernal, Matteo Bertrone, and Yunsong Lu. A framework for ebpf-based network functions in an era of microservices. *IEEE Transactions on Network and Service Management*, 18(1):133–151, 2021.

[40] Andrii Nakryiko. BPF CO-RE reference guide, 2021.

[41] Usama Naseer and Theophilus A. Benson. Configanator: A data-driven approach to improving CDN performance. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1135–1158, Renton, WA, April 2022. USENIX Association.

[42] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. Scaling symbolic evaluation for automated verification of systems code with serval. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 225–242, New York, NY, USA, 2019. Association for Computing Machinery.

[43] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the linux kernel. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 41–61, 2020.

[44] Xiaohui Nie, Youjian Zhao, Dan Pei, Guo Chen, Kaixin Sui, and Jiyang Zhang. Reducing web latency through dynamically setting TCP initial window with reinforcement learning. In *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*, pages 1–10, 2018.

[45] Sujin Park, Diyu Zhou, Yuchen Qian, Irina Calciu, Taesoo Kim, and Sanidhya Kashyap. Application-Informed kernel synchronization primitives. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 667–682, 2022.

[46] Joshua Reich, Christopher Monsanto, Nate Foster, Jennifer Rexford, and David Walker. Modular SDN programming with Pyretic. *;login:*, 38(5), October 2013.

[47] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. Inside the social network's (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 123–137, 2015.

[48] Brandon Schlinker, Hyojeong Kim, Timothy Cui, Ethan Katz-Bassett, Harsha V Madhyastha, Italo Cunha, James Quinn, Saif Hasan, Petr Lapukhov, and Hongyi Zeng. Engineering egress with edge fabric: Steering oceans of content to the world. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 418–431, 2017.

[49] Itay Shakury and Rafael David Tinoco. Towards truly portable eBPF. In *Linux Plumbers Conference*, 2021.

[50] Junxian Shen, Han Zhang, Yang Xiang, Xingang Shi, Xinrui Li, Yunxi Shen, Zijian Zhang, Yongxiang Wu, Xia Yin, Jilong Wang, et al. Network-centric distributed tracing with deepflow: Troubleshooting your microservices in zero code. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 420–437, 2023.

[51] Viet-Hoang Tran and Olivier Bonaventure. Making the linux TCP stack more extensible with eBPF. In *Proc. of the Netdev 0x13, Technical Conference on Linux Networking*, 2019.

[52] Kaushik Veeraraghavan, Justin Meza, David Chou, Wonho Kim, Sonia Margulis, Scott Michelson, Rajesh Nishtala, Daniel Obenshain, Dmitri Perelman, and Yee Jiun Song. Kraken: Leveraging live traffic tests to identify and resolve resource utilization bottlenecks in large scale web services. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 635–651, Savannah, GA, November 2016. USENIX Association.

[53] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Verifying the verifier: eBPF range analysis verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III*, page 226–251, Berlin, Heidelberg, 2023. Springer-Verlag.

[54] l3af. Complete lifecycle management of eBPF programs in the kernel. https://l3af.io/, 2024.

[55] Sepehr Abbasi Zadeh, Ali Munir, Mahmoud Mohamed Bahnasy, Shiva Ketabi, and Yashar Ganjali. On augmenting TCP/IP stack via eBPF. In *Proceedings of the 1st Workshop on EBPF and Kernel Extensions*, eBPF '23, page 15–20, New York, NY, USA, 2023. Association for Computing Machinery.

[56] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, et al. XRP:In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, 2022.

[57] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. Electrode: Accelerating distributed protocols with eBPF. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1391–1407, 2023.

Appendices are supporting material that has not been peer-reviewed.

A COMPARISON OF APPROACHES FOR TUNING THE KERNEL

There exists a broad set of techniques for tuning the Linux kernel network stack (e.g., eBPF programs, kernel modules, SetSockOpts, IPTables), these techniques offer different design trade offs and levels of flexibility. On one hand, writing a kernel module offers significant flexibility with regards to the granularity of tuning and coverage over functionality being tuned; however, this comes at the cost of limited portability – specifically, it is challenging to use the same modules across different kernel versions due to kernel code changes. On the other end of the spectrum, primitives like SetSockOpts and IPTables are portable across kernel versions; however, they provide limited flexibility and coverage. For example, SetSockOpts enables tuning only at the granularity of connections where-as IPTables does not support tuning of transport layer functionality [9]. Moreover, certain options, e.g., SetSockOpts, must be designed as libraries which are ultimately coupled with the applications.

eBPF has recently emerged as an arbitrarily flexible approach to tuning arbitrary components within the kernel which does not limit portability nor introduce unnecessary coupling. It is precisely because of eBPF's flexibility, broad coverage, and portability, that many large clouds and CDNs [7, 17] have adopted eBPF for realizing network functions amongst other service-specific tailoring of the Linux kernel.

Another promising alternative, kernel-bypass and userspace network stacks, offer similar benefits as eBPF but requires the upfront cost of rewriting/redesigning the userspace stack whereas eBPF allows for reuse of hardened and efficient kernel implementations.

[9] Similarly, IPTables enables tuning at the granularity of IP addresses (and ports) which may not be granular enough for contextual tuning